Laboratorio 10 Componentes y eventos

Desarrollo de Aplicaciones Empresariales

Estudiante: Pamela Meneses | Docente: Michael Montgomery Rosell | Fecha: 30 de octubre de 2026

Se amplió la cartelera del laboratorio 9 con Button reutilizable, búsqueda controlada con retardo, favoritas y persistencia en el navegador. El paso 1 de la guía exige continuar el proyecto anterior.

El informe sigue los doce pasos de GLAB-S10-MMONTGOMERY-2026-02.docx. Se conserva la cartelera de ocho películas y el diseño base.

1 Partir del laboratorio 9

Estudiante: Pamela Meneses.

Se continuó el frontend existente de GLAB-S03-Meneses, con su cartelera de ocho películas, datos, estilos y componentes. No se creó otro proyecto.

Resultado: Inicio con ocho tarjetas y servidor Vite en el puerto 5173.

Caso de prueba: Abrir la página y contar ocho películas.

```
cd frontend
npm run dev

# Verification
npm run build
npm run lint
```

2 Ampliar los estilos base

Estudiante: Pamela Meneses.

Se añadieron estilos para entradas de búsqueda, tipografía de controles, texto auxiliar y foco visible. Los estilos de Button y MovieSearch se mantienen en sus carpetas.

Resultado: Controles legibles e integrados en el diseño existente.

Caso de prueba: Revisar formulario en escritorio y móvil; no hay desbordamiento a 390 px.

Código: frontend/src/styles/global.css

```
:root { font-family: Arial, sans-serif; color: #eef1f6; background: #0a1019; font-synthesis: none; text-
    rendering: optimizeLegibility; --accent: #efc17c; --muted: #9ca7b7; --border: #252e3a; }
* { box-sizing: border-box; }
html { scroll-behavior: smooth; }
body { margin: 0; min-width: 320px; }
h1, h2, h3 { font-weight: 700; }
a { color: inherit; }
a:focus-visible, button:focus-visible, input:focus-visible { outline: 2px solid var(--accent); outline-
    offset: 5px; }
input, button { font: inherit; }
input[type='search'] { padding: 13px 14px; color: #eef1f6; background: #111c2c; border: 1px solid
    #526076; border-radius: 5px; font-size: 14px; }
input::placeholder { color: #a5b0c0; }
.form-hint { color: var(--muted); font-size: 11px; line-height: 1.5; margin: 9px 0 0; }
@media (prefers-reduced-motion: reduce) { html { scroll-behavior: auto; } *, *::before, *::after {
    transition: none !important; } }
```

3 Crear Button reutilizable

Estudiante: Pamela Meneses.

Button recibe contenido mediante children y aspecto mediante variant. Las variantes primary, secondary y selected comparten una única implementación. El buscador utiliza primary; cada tarjeta utiliza secondary o selected según su estado. La llamada a la acción anterior del banner se sustituyó por el botón del formulario.

Resultado: El mismo componente sirve en el banner y en ocho tarjetas cambiando props.

Caso de prueba: Buscar una película y marcar una favorita: ambos controles utilizan Button.

Código: frontend/src/components/Button/Button.jsx

```
import './Button.css'

export default function Button({ children, variant = 'primary', type = 'button', className = '', ...props
     }) {
  return (
    <button type={type} className={'button button--' + variant + ' ' + className} {...props}>
      {children}
    </button>
  )
}

<Button type="submit" variant="primary">Buscar películas</Button>
<Button variant={isFavorite ? "selected" : "secondary"}>
  {isFavorite ? "En favoritas" : "Agregar a favoritas"}
</Button>
```

4 Añadir el estado de favoritas a App

Estudiante: Pamela Meneses.

App mantiene favoriteIds mediante useState. toggleFavorite usa la actualización funcional para agregar o quitar un identificador sin mutar el arreglo anterior. Se guardan identificadores, no copias completas de las películas.

Resultado: Una sola fuente del estado de favoritas, compartida por todas las tarjetas.

Caso de prueba: Marcar dos películas: el contador muestra dos favoritas.

Código: frontend/src/App.jsx

```
import { useEffect, useState } from 'react'
import Header from './components/Header/Header.jsx'
import Footer from './components/Footer/Footer.jsx'
import Hero from './components/Hero/Hero.jsx'
import MovieList from './components/MovieList/MovieList.jsx'
import { movies } from './data/movies.js'

const FAVORITES_KEY = 'orbita.favoriteMovieIds'

function loadFavoriteIds() {
  try {
    const savedIds = JSON.parse(localStorage.getItem(FAVORITES_KEY) ?? '[]')
    return Array.isArray(savedIds)
      ? [...new Set(savedIds.filter((id) => movies.some((movie) => movie.id === id)))]
      : []
  } catch {
    return []
  }
}

export default function App() {
  const [favoriteIds, setFavoriteIds] = useState(loadFavoriteIds)
  const [searchQuery, setSearchQuery] = useState('')

  useEffect(() => {
    try {
      localStorage.setItem(FAVORITES_KEY, JSON.stringify(favoriteIds))
    } catch {
```

```
      console.warn('Favorite movies could not be saved in this browser.')
    }
  }, [favoriteIds])

  function toggleFavorite(movieId) {
    setFavoriteIds((currentIds) => currentIds.includes(movieId)
      ? currentIds.filter((id) => id !== movieId)
      : [...currentIds, movieId])
  }

  return (
    <>
      <Header />
      <main>
        <Hero movie={movies[0]} onSearch={setSearchQuery} />
        <MovieList
          movies={movies}
          searchQuery={searchQuery}
          favoriteIds={favoriteIds}
          onToggleFavorite={toggleFavorite}
        />
      </main>
      <Footer />
    </>
  )
}
```

5 Comunicar MovieCard con App

Estudiante: Pamela Meneses.

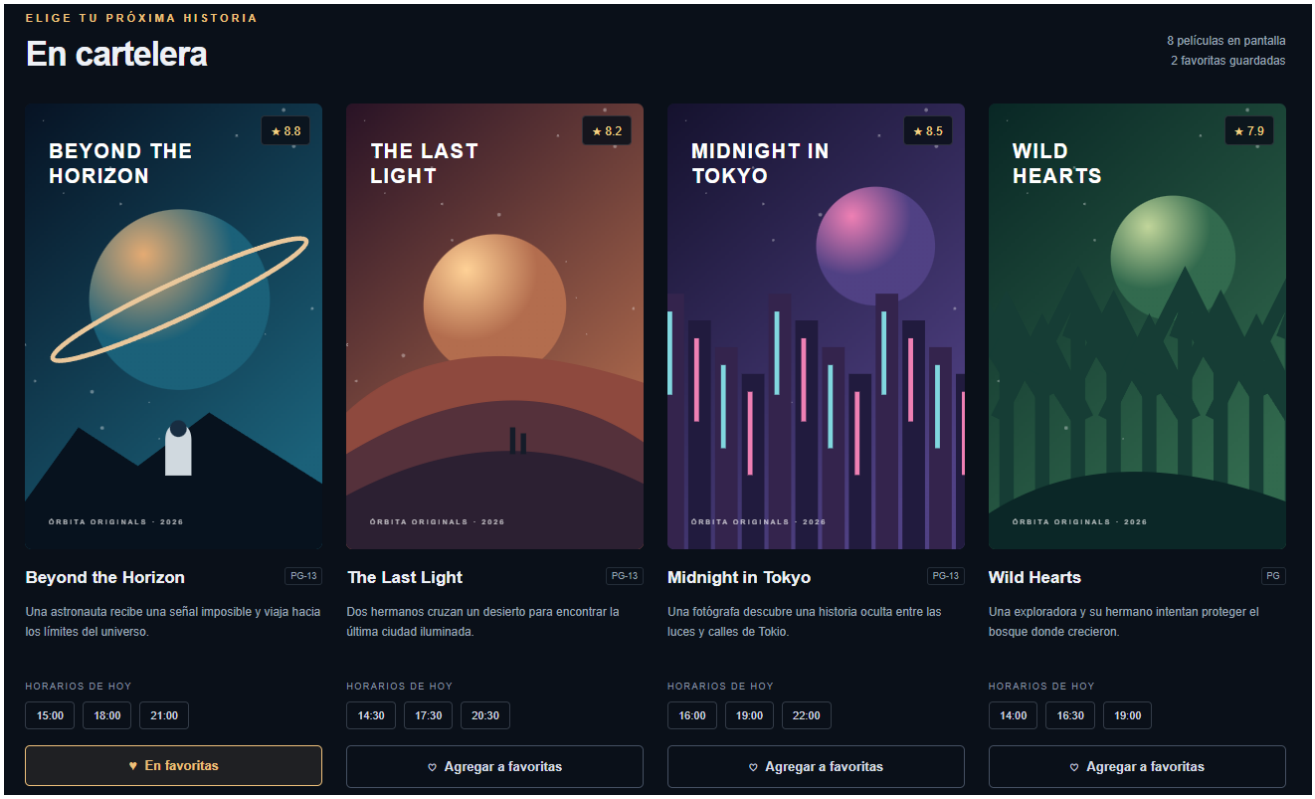
App pasa toggleFavorite como onToggleFavorite a MovieList y luego a MovieCard. La tarjeta llama onToggleFavorite(movie.id) al pulsar el botón; App actualiza el estado y devuelve isFavorite. El hijo no modifica el estado del padre.

Resultado: El botón alterna entre Agregar a favoritas y En favoritas, con aria-pressed y variante visual.

Caso de prueba: Marcar y desmarcar la primera película: cambia el texto, el estilo y el contador sin recargar.

Código: frontend/src/components/MovieCard/MovieCard.jsx

```
import Button from '../Button/Button.jsx'
import './MovieCard.css'
export default function MovieCard({ movie, isFavorite, onToggleFavorite }) {
  return (
    <article className="movie-card">
      <div className="poster-wrap">
        <img src={movie.image} alt={movie.title} loading="lazy" />
        <span className="movie-rating" aria-label={'Valoración: ' + movie.rating + ' de 10'}>
    {movie.rating.toFixed(1)}</span>
      </div>
      <div className="movie-title-row"><h3>{movie.title}</h3><span
    className="classification">{movie.classification}</span></div>
      <p className="movie-synopsis">{movie.synopsis}</p>
      <p className="showtimes-label">HORARIOS DE HOY</p>
      <ul className="showtimes" aria-label={'Horarios de ' + movie.title}>
        {movie.showtimes.map((showtime) => <li key={showtime}><time>{showtime}</time></li>)}
      </ul>
      <Button
        className="favorite-button"
        variant={isFavorite ? 'selected' : 'secondary'}
        aria-pressed={isFavorite}
        aria-label={(isFavorite ? 'Quitar de favoritas: ' : 'Agregar a favoritas: ') + movie.title}
        onClick={() => onToggleFavorite(movie.id)}
      >
        <span aria-hidden="true">{isFavorite ? '♥' : ' '}</span>
        {isFavorite ? 'En favoritas' : 'Agregar a favoritas'}
      </Button>
    </article>
  )
}
```



Captura real después de recargar: la primera película permanece En favoritas y el contador indica dos guardadas.

6 Crear MovieSearch controlado

Estudiante: Pamela Meneses.

searchText vive en MovieSearch con useState. El valor del input viene de ese estado y onChange actualiza cada pulsación. El formulario evita la recarga con preventDefault.

Resultado: La entrada siempre refleja el estado de React.

Caso de prueba: Escribir Tokyo: la entrada refleja las cinco letras inmediatamente.

Código: frontend/src/components/MovieSearch/MovieSearch.jsx

```
import { useEffect, useRef, useState } from 'react'
import Button from '../Button/Button.jsx'
import './MovieSearch.css'

export default function MovieSearch({ onSearch }) {
  const [searchText, setSearchText] = useState('')
  const timeoutRef = useRef(null)

  useEffect(() => {
    timeoutRef.current = setTimeout(() => {
      console.info('[search:filter]', searchText.trim())
      onSearch(searchText.trim())
    }, 400)

    return () => clearTimeout(timeoutRef.current)
  }, [searchText, onSearch])

  function handleChange(event) {
    const nextText = event.target.value
    console.info('[search:input]', nextText)
    setSearchText(nextText)
  }

  function handleSubmit(event) {
    event.preventDefault()
    clearTimeout(timeoutRef.current)
    console.info('[search:submit]', searchText.trim())
    onSearch(searchText.trim())
```

```
    window.location.hash = 'movies'
  }

  return (
    <form className="movie-search" role="search" onSubmit={handleSubmit}>
      <label htmlFor="movie-search">Encuentra tu próxima película</label>
      <div className="search-controls">
        <input
          id="movie-search"
          type="search"
          placeholder="Buscar por título…"
          value={searchText}
          onChange={handleChange}
          aria-describedby="search-hint"
        />
        <Button type="submit" variant="primary">Buscar películas <span aria-
    hidden="true"> </span></Button>
      </div>
      <p className="form-hint" id="search-hint">La cartelera se actualiza al terminar de escribir.</p>
    </form>
  )
}
```

7 Añadir el retardo de búsqueda

Estudiante: Pamela Meneses.

Un efecto programa onSearch tras 400 ms y cancela el temporizador anterior en cada cambio. La referencia timeoutRef guarda el temporizador sin provocar renders. La consola diferencia [search:input] de [search:filter]. Al enviar el formulario se cancela el retardo pendiente y se aplica la búsqueda inmediatamente.

Resultado: Prueba real con Tokyo: cinco eventos de entrada y un filtrado después de la pausa.

Caso de prueba: Escribir Tokyo rápido y observar la consola: T, To, Tok, Toky, Tokyo y un único [search:filter] Tokyo.

```
[search:input] T
[search:input] To
[search:input] Tok
[search:input] Toky
[search:input] Tokyo
[search:filter] Tokyo
```

Registro real de la consola capturado en la prueba: cinco entradas y un filtrado, con 400 ms de pausa. Los mensajes iniciales se excluyeron del conteo.

8 Filtrar MovieList y mostrar estado vacío

Estudiante: Pamela Meneses.

MovieList recibe searchQuery y filtra por título sin distinguir mayúsculas. filteredMovies se calcula a partir de props, no se duplica en estado. Sin coincidencias se muestran cero películas y un mensaje explicativo.

Resultado: Tokyo devuelve Midnight in Tokyo; una consulta inexistente muestra el estado vacío.

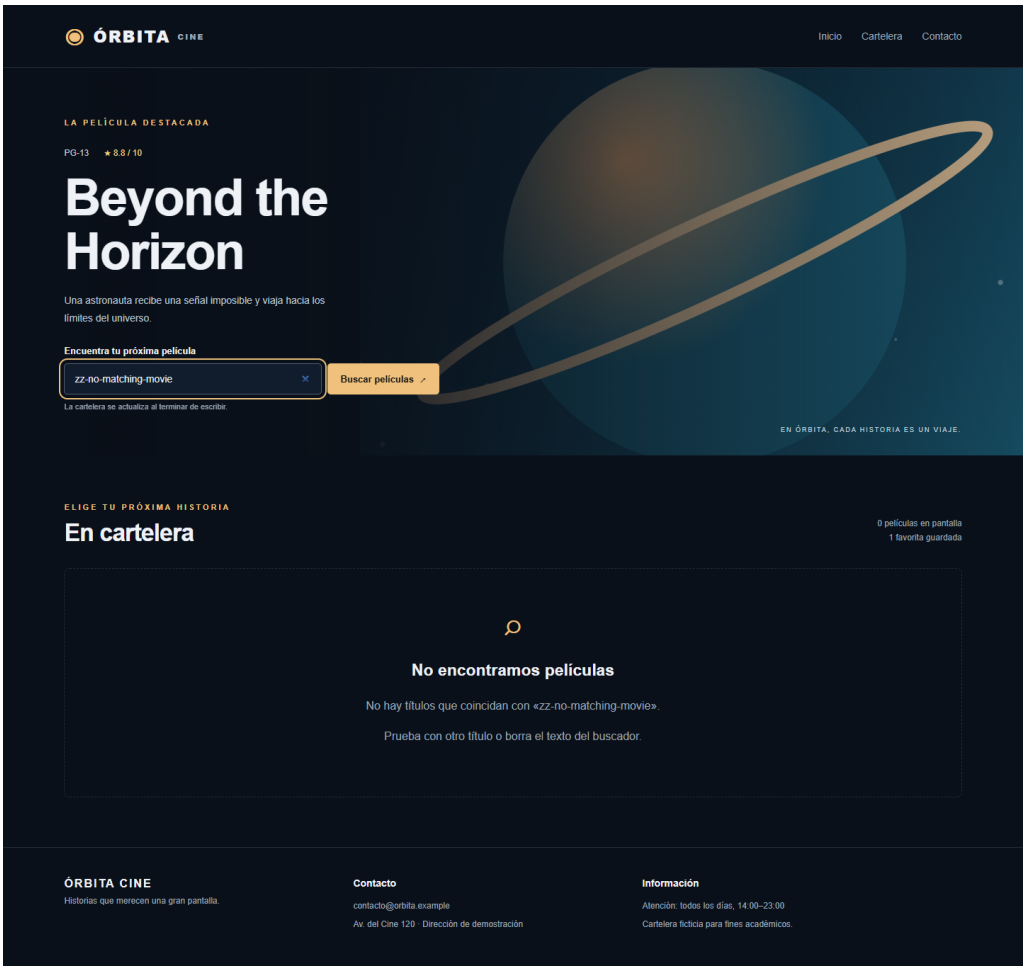
Caso de prueba: Buscar zz-no-matching-movie: aparece No encontramos películas y no hay tarjetas. Borrar el texto restaura ocho.

Código: frontend/src/components/MovieList/MovieList.jsx

```
import MovieCard from '../MovieCard/MovieCard.jsx'
import './MovieList.css'
export default function MovieList({ movies, searchQuery, favoriteIds, onToggleFavorite }) {
  const normalizedQuery = searchQuery.trim().toLowerCase()
  const filteredMovies = movies.filter((movie) => movie.title.toLowerCase().includes(normalizedQuery))

  return (
    <section className="movie-list" id="movies" aria-labelledby="movies-title">
      <div className="section-heading">
```

```
        <div><p className="eyebrow">ELIGE TU PRÓXIMA HISTORIA</p><h2 id="movies-title">En
    cartelera</h2></div>
        <div className="list-summary" role="status" aria-live="polite">
          <p>{filteredMovies.length} {filteredMovies.length === 1 ? 'película en pantalla' : 'películas
    en pantalla'}</p>
          <p>{favoriteIds.length} {favoriteIds.length === 1 ? 'favorita guardada' : 'favoritas
    guardadas'}</p>
        </div>
      </div>
      {filteredMovies.length > 0 ? (
        <div className="movie-grid">
          {filteredMovies.map((movie) => (
            <MovieCard
              key={movie.id}
              movie={movie}
              isFavorite={favoriteIds.includes(movie.id)}
              onToggleFavorite={onToggleFavorite}
            />
          ))}
        </div>
      ) : (
        <div className="empty-state" role="status">
          <span aria-hidden="true">  </span>
          <h3>No encontramos películas</h3>
          <p>No hay títulos que coincidan con «{searchQuery}».</p>
          <p>Prueba con otro título o borra el texto del buscador.</p>
        </div>
      )}
    </section>
  )
}
```



Estado vacío real solicitado por la guía: consulta zz-no-matching-movie, cero películas y mensaje explicativo.

9 Integrar el buscador en Hero

Estudiante: Pamela Meneses.

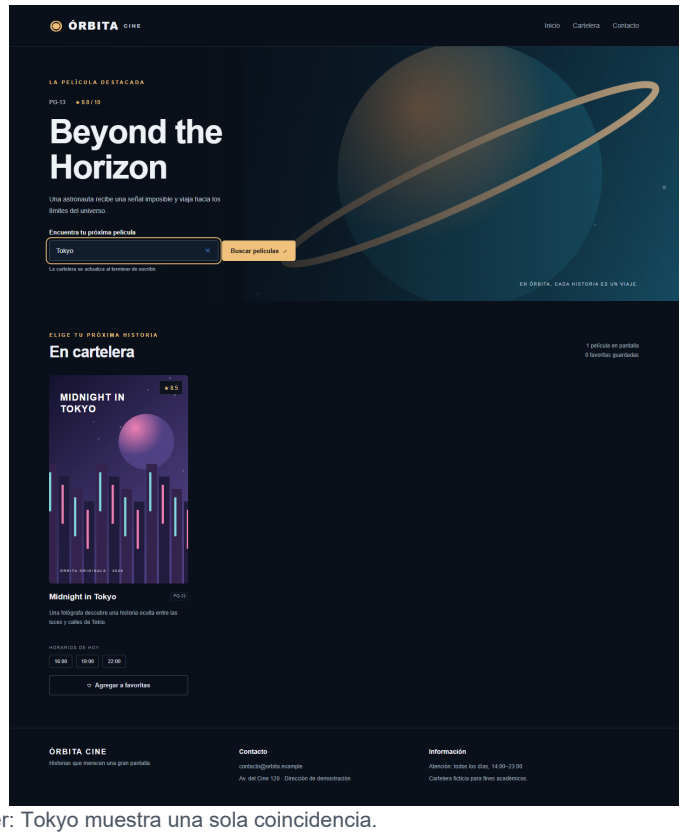
Hero recibe onSearch desde App y lo entrega a MovieSearch. El botón Buscar películas envía el formulario, aplica el texto y navega a #movies. App conserva la consulta aplicada para compartirla entre el banner y la lista.

Resultado: La búsqueda funciona desde el banner y su llamada a la acción.

Caso de prueba: Buscar oCeAn y pulsar el botón: aparece Ocean of Secrets y la dirección termina en #movies.

Código: frontend/src/components/Hero/Hero.jsx

```
import MovieSearch from '../MovieSearch/MovieSearch.jsx'
import './Hero.css'
export default function Hero({ movie, onSearch }) {
  return (
    <section className="hero" id="home" aria-labelledby="featured-title">
      <img className="hero-image" src={movie.image} alt={movie.title} />
      <div className="hero-content">
        <p className="eyebrow">LA PELÍCULA DESTACADA</p>
        <p className="hero-meta">{movie.classification} <span>  {movie.rating.toFixed(1)} / 10</span></p>
        <h1 id="featured-title">{movie.title}</h1>
        <p className="hero-synopsis">{movie.synopsis}</p>
        <MovieSearch onSearch={onSearch} />
      </div>
      <p className="hero-caption">EN ÓRBITA, CADA HISTORIA ES UN VIAJE.</p>
    </section>
  )
}
```



Buscador integrado en el banner: Tokyo muestra una sola coincidencia.

10 Persistir y recuperar favoritas

Estudiante: Pamela Meneses.

App inicializa favoriteIds leyendo localStorage mediante un inicializador de useState. Un efecto guarda cada cambio en orbita.favoriteMovieIds. Se valida el formato, se descartan ids inexistentes y se evitan duplicados. Ante un dato inválido se utiliza una lista vacía.

Resultado: Las favoritas permanecen tras recargar el mismo navegador y origen.

Caso de prueba: Guardar las películas 1 y 8, recargar y comprobar ambos botones seleccionados. Quitar la 1 y comprobar que queda el id 8.

11 Documentar la ubicación de cada dato

Estudiante: Pamela Meneses.

Se documentó por qué favoriteIds y searchQuery viven en App, por qué searchText y timeoutRef viven en MovieSearch, y por qué los datos derivados no requieren useState. Se incluyó la captura real del estado vacío.

Resultado: Decisiones de estado documentadas y evidencias organizadas.

Caso de prueba: Revisar la tabla de estado y evidence/empty-state.png.

favoriteIds - App / useState. Lo comparten todas las tarjetas y el contador; App coordina sus cambios. Persistencia: Sí, localStorage.

searchQuery - App / useState. Consulta aplicada después del retardo o del envío; conecta el buscador del banner con MovieList. Persistencia: No.

searchText - MovieSearch / useState. Texto inmediato de la entrada controlada, distinto de la consulta aplicada durante el retardo. Persistencia: No.

timeoutRef - MovieSearch / useRef. Identificador del temporizador para cancelarlo; no afecta por sí mismo a la interfaz. Persistencia: No.

filteredMovies - MovieList / dato derivado. Se calcula desde movies y searchQuery; guardarlo en estado duplicaría información. Persistencia: No.

isFavorite - MovieList / prop derivada. Se calcula con favoriteIds.includes(movie.id) y se entrega a la tarjeta. Persistencia: No.

movies - src/data/movies.js / constante. Cartelera del laboratorio 9; no cambia durante esta práctica. Persistencia: Archivo del proyecto.

searchText y searchQuery representan momentos distintos: la entrada actual y la consulta ya aplicada. No son dos copias del mismo estado de favoritas. filteredMovies e isFavorite se derivan de datos existentes.

Las favoritas se guardan por origen. localhost:5173 y 127.0.0.1:5173 tienen almacenamientos independientes; la prueba debe repetirse usando siempre la misma dirección.

12 Preparar repositorio y entrega

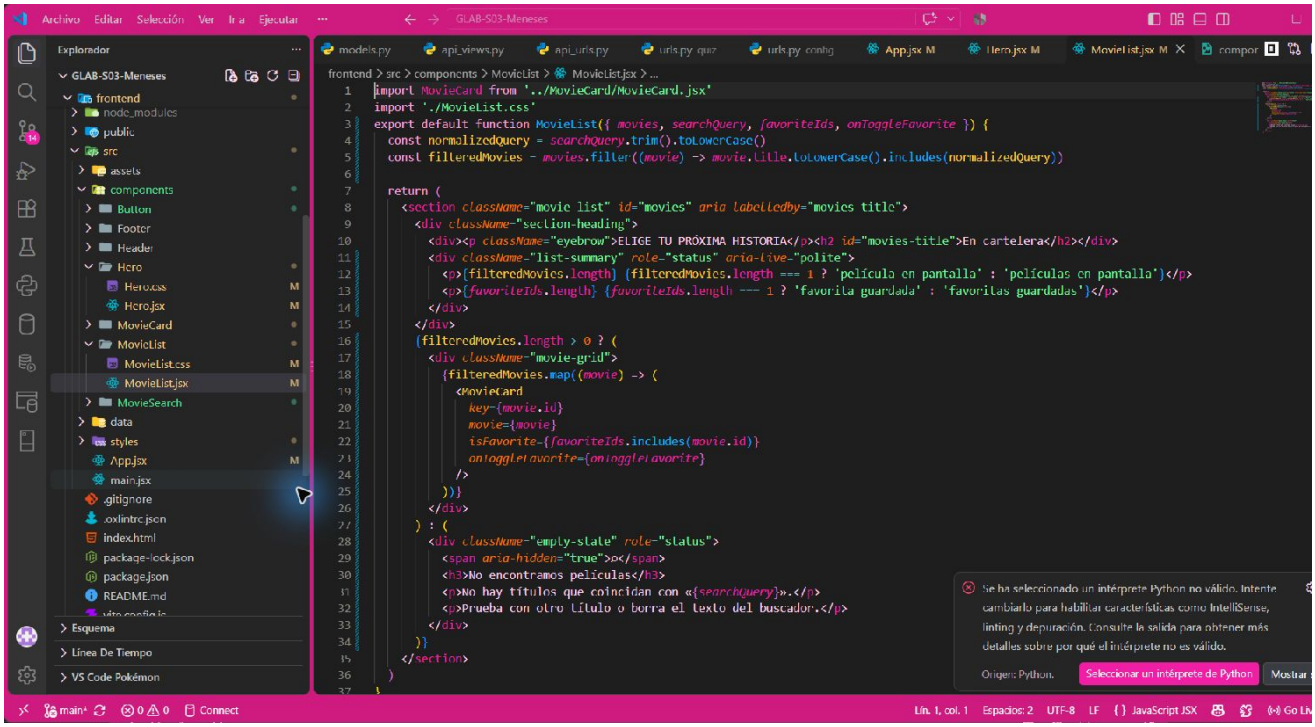
Estudiante: Pamela Meneses.

La continuación se guarda en el mismo repositorio. Los componentes y sus observaciones se documentan en docs/lab10. Se incluyen código, pruebas, capturas y dependencias ya declaradas en package.json; no se añadieron librerías.

Resultado: Código y evidencias listos para revisión.

Caso de prueba: Ejecutar npm run build y npm run lint. El equipo debe completar aportes reales y presentar el informe en el campus dentro del plazo.

Estructura del proyecto en el editor



Captura real de VS Code. Se observan Button, MovieSearch, los componentes del laboratorio 9 y el paso de callbacks a las tarjetas.

Flujo de eventos y datos

```
App [favoriteIds, searchQuery]
 |-- Hero
 |    |-- MovieSearch [searchText, timeoutRef]
 |         |-- Button [children, variant="primary"]
 |         |-- onSearch -> App updates searchQuery
 |-- MovieList [filteredMovies derived from props]
      |-- MovieCard [movie, isFavorite, onToggleFavorite]
           |-- Button [children, variant]
           |-- onClick -> onToggleFavorite(movie.id)
                       -> App updates favoriteIds
```

Verificación realizada

Se superaron 19 comprobaciones funcionales en un navegador aislado. No se modificaron las favoritas personales del navegador del estudiante. checks.json contiene cada resultado.

Se verificaron ocho tarjetas iniciales, imágenes cargadas, retardo de 400 ms, una coincidencia para Tokyo, altas y bajas de favoritas, recuperación tras recarga, llamada a la acción desde el banner, búsquedas sin distinguir mayúsculas, estado vacío, recuperación al borrar, ausencia de desbordamiento móvil y consola sin errores ni

advertencias. La compilación y la revisión de código terminaron correctamente.

Observaciones sobre reutilización

Button tiene una sola implementación. children permite cambiar el texto y los iconos; variant cambia el aspecto mediante CSS. Las props restantes permiten reutilizar eventos, tipo de botón y atributos de accesibilidad. MovieCard comunica el evento al padre sin modificar un estado ajeno.

Conclusiones

Button permite reutilizar controles con contenido y variantes por props.

useState mantiene las favoritas y la entrada controlada sincronizadas con la interfaz.

Los callbacks permiten comunicar eventos desde las tarjetas hacia App.

El retardo reduce los filtrados durante la escritura y el estado vacío informa cuando no hay coincidencias.

localStorage conserva las favoritas entre recargas del mismo navegador y origen.

Integrantes y entrega

Estudiante: Pamela Meneses. Completar la distribución real de aportes entre integrantes antes de presentar; no se han supuesto nombres ni responsabilidades.

Repositorio del equipo: https://github.com/pamelameneses-code/GLAB-S03-Meneses-D.E

Código: frontend/. Evidencias y observaciones: docs/lab10/. El equipo debe presentar el entregable en el campus virtual dentro del plazo de la semana.

Referencia: guía GLAB-S10-MMONTGOMERY-2026-02.docx proporcionada para el curso.